TITAN TRUST: COMPREHENSIVE COMPLIANCE & ASSET MANAGEMENT APP UPGRADE PLAN
Full Enterprise Compliance, Risk, Assets, Fleet, Security & Document Management System
System: Titan Trust (Part of TitanBOS - Titan Business Operating System)AI Core: Titan Zero (Cognitive decision engine)Timeline: 22 weeks (5.5 months)Team: 3 developers + 1 designer + 1 QATotal LOC: 28,000-32,000 lines addedPlatforms: iOS + Android (Flutter)Target Users: Safety officers, compliance managers, fleet managers, business owners, legal teams

PHASE 0: FOUNDATION (Weeks 1-2)
Security + Comprehensive Document Framework
0.1 Critical Security (4 hours)
[Same as before: API keys, token encryption, certificate pinning, audit logging]
0.2 Universal Document Framework (25 hours)

```
// Abstract base for ALL document types
abstract class ComplianceDocument {
  final String id;
  final String documentType; // swms, incident, permit, asset, fleet, contract, insurance, policy, etc
  final String title;
  final String description;
  final DateTime createdDate;
  final DateTime? expiryDate;
  final String createdBy;
  final String? approvedBy;
  final DateTime? approvedDate;
  final DocumentStatus status;

  // Audit trail
  final List<AuditEntry> auditTrail;

  // Digital signature
  final String? digitalSignature;
  final String? signatureHash;

  // Encryption & storage
  final bool isEncrypted;
  final String? storageLocation; // s3, azure, gcp, etc (user's account)

  // Tags for searching
  final List<String> tags;
  final Map<String, dynamic> metadata;

  ComplianceDocument({
```

```
    required this.id,
    required this.documentType,
    required this.title,
    required this.description,
    required this.createdDate,
    this.expiryDate,
    required this.createdBy,
    this.approvedBy,
    this.approvedDate,
    required this.status,
    required this.auditTrail,
    this.digitalSignature,
    this.signatureHash,
    this.isEncrypted = true,
    this.storageLocation,
    this.tags = const [],
    this.metadata = const {},
  });
}

enum DocumentStatus {
  draft,
  submitted,
  under_review,
  approved,
  active,
  expiring_soon,
  expired,
  archived,
  revoked,
  renewal_in_progress,
}

class AuditEntry {
  final String id;
  final String action; // created, modified, approved, viewed, signed, downloaded, shared
  final String userId;
  final String userName;
  final DateTime timestamp;
  final String? details;
  final String ipAddress;
  final String deviceInfo;
  final String? changesSummary; // What changed
```

```
  const AuditEntry({
    required this.id,
    required this.action,
    required this.userId,
    required this.userName,
    required this.timestamp,
    this.details,
    required this.ipAddress,
    required this.deviceInfo,
    this.changesSummary,
  });
}

// Universal document repository
class DocumentRepository extends GetxService {
  final database = Get.find<OfflineFirstDatabase>();
  final auditLogger = Get.find<AuditLoggingService>();
  final storageService = Get.find<StorageService>(); // User's cloud storage

  Future<void> saveDocument(ComplianceDocument doc) async {
    // Serialize document
    final docJson = _serializeDocument(doc);

    // Encrypt if required
    String docContent = jsonEncode(docJson);
    if (doc.isEncrypted) {
      docContent = await _encryptContent(docContent);
    }

    // Save to local database
    await database.db.insert('documents', {
      'id': doc.id,
      'document_type': doc.documentType,
      'title': doc.title,
      'description': doc.description,
      'created_by': doc.createdBy,
      'created_at': doc.createdDate.toIso8601String(),
      'expiry_date': doc.expiryDate?.toIso8601String(),
      'status': doc.status.name,
      'content': docContent,
      'encrypted': doc.isEncrypted ? 1 : 0,
      'tags': jsonEncode(doc.tags),
      'metadata': jsonEncode(doc.metadata),
      'digital_signature': doc.digitalSignature,
```

```
    'synced': 0,
  });

  // Upload to cloud storage if online
  if (await networkInfo.isConnected()) {
    final fileUrl = await storageService.uploadDocument(
      fileName: '${doc.documentType}_${doc.id}.json',
      fileContent: docContent,
      documentType: doc.documentType,
    );

    // Update storage location
    await database.db.update(
      'documents',
      {'storage_location': fileUrl},
      where: 'id = ?',
      whereArgs: [doc.id],
    );
  }
}

Future<ComplianceDocument?> getDocument(String docId) async {
  final results = await database.db.query(
    'documents',
    where: 'id = ?',
    whereArgs: [docId],
  );

  if (results.isEmpty) return null;

  final row = results.first;

  var content = row['content'] as String;

  // Decrypt if needed
  if (row['encrypted'] == 1) {
    content = await _decryptContent(content);
  }

  final docJson = jsonDecode(content);
  return _deserializeDocument(docJson);
}

Future<List<ComplianceDocument>> searchDocuments({
```

```
  String? query,
  String? documentType,
  DocumentStatus? status,
  DateTime? createdAfter,
  DateTime? createdBefore,
  String? tag,
}) async {
  var dbQuery = 'SELECT * FROM documents WHERE 1=1';
  final args = <dynamic>[];

  if (query != null && query.isNotEmpty) {
    dbQuery += ' AND (title LIKE ? OR description LIKE ?)';
    args.addAll(['%$query%', '%$query%']);
  }

  if (documentType != null) {
    dbQuery += ' AND document_type = ?';
    args.add(documentType);
  }

  if (status != null) {
    dbQuery += ' AND status = ?';
    args.add(status.name);
  }

  if (createdAfter != null) {
    dbQuery += ' AND created_at >= ?';
    args.add(createdAfter.toIso8601String());
  }

  if (createdBefore != null) {
    dbQuery += ' AND created_at <= ?';
    args.add(createdBefore.toIso8601String());
  }

  final results = await database.db.rawQuery(dbQuery, args);

  final documents = <ComplianceDocument>[];

  for (final row in results) {
    var content = row['content'] as String;

    if (row['encrypted'] == 1) {
      content = await _decryptContent(content);
```

```
    }

    final docJson = jsonDecode(content);
    final doc = _deserializeDocument(docJson);

    if (doc != null) {
      // Apply tag filter if needed
      if (tag != null && !doc.tags.contains(tag)) {
        continue;
      }

      documents.add(doc);
    }
  }

  return documents;
}

Future<List<ComplianceDocument>> getExpiringDocuments({
  int daysUntilExpiry = 30,
}) async {
  final expiryDate = DateTime.now().add(Duration(days: daysUntilExpiry));

  return searchDocuments(
    createdBefore: expiryDate,
    status: DocumentStatus.active,
  );
}

String _serializeDocument(ComplianceDocument doc) {
  return jsonEncode({
    'id': doc.id,
    'documentType': doc.documentType,
    'title': doc.title,
    'description': doc.description,
    'createdDate': doc.createdDate.toIso8601String(),
    'expiryDate': doc.expiryDate?.toIso8601String(),
    'createdBy': doc.createdBy,
    'approvedBy': doc.approvedBy,
    'approvedDate': doc.approvedDate?.toIso8601String(),
    'status': doc.status.name,
    'digitalSignature': doc.digitalSignature,
    'tags': doc.tags,
    'metadata': doc.metadata,
```

```
    });
  }

  ComplianceDocument? _deserializeDocument(Map<String, dynamic> json) {
    // Factory method would call specific document type constructors
    // For now, return generic
    return null;
  }

  Future<String> _encryptContent(String content) async {
    // Encrypt using user's encryption key
    final encrypter = Get.find<EncryptionService>();
    return await encrypter.encryptAES256(content);
  }

  Future<String> _decryptContent(String encrypted) async {
    final encrypter = Get.find<EncryptionService>();
    return await encrypter.decryptAES256(encrypted);
  }
}

// Universal document database schema
void _createDocumentTables(Database db, int version) async {
  // Universal documents table
  await db.execute('''
    CREATE TABLE documents (
      id TEXT PRIMARY KEY,
      document_type TEXT NOT NULL,
      title TEXT NOT NULL,
      description TEXT,
      created_by TEXT NOT NULL,
      created_at TEXT NOT NULL,
      expiry_date TEXT,
      status TEXT NOT NULL,
      content TEXT NOT NULL,
      encrypted INTEGER DEFAULT 1,
      storage_location TEXT,
      tags TEXT,
      metadata TEXT,
      digital_signature TEXT,
      synced INTEGER DEFAULT 0
    );
  ''');
```

```
  // Create indices for fast searching
  await db.execute('CREATE INDEX idx_docs_type ON documents(document_type)');
  await db.execute('CREATE INDEX idx_docs_status ON documents(status)');
  await db.execute('CREATE INDEX idx_docs_expiry ON documents(expiry_date)');
  await db.execute('CREATE INDEX idx_docs_created ON documents(created_at)');

  // Full text search
  await db.execute('''
    CREATE VIRTUAL TABLE documents_fts USING fts5(
      title,
      description,
      tags
    );
  ''');
}
```

PHASE 1: SWMS & SAFETY DOCUMENTATION (Weeks 3-5)
Safe Work Method Statements, JSA, Risk Assessments, OHS Plans
[See previous SWMS section - 3 hours]
Plus additions:

```
class SafetyPlan extends ComplianceDocument {
  final String planType; // ohs_plan, risk_register, incident_response_plan, emergency_plan
  final String scope;
  final List<String> responsibilities; // Who is responsible for what
  final List<SafetyObjective> objectives;
  final DateTime reviewDate;
  final List<String> relatedSwmsIds;

  SafetyPlan({
    required String id,
    required String title,
    required String description,
    required DateTime createdDate,
    required String createdBy,
    required DocumentStatus status,
    required List<AuditEntry> auditTrail,
    required this.planType,
    required this.scope,
    required this.responsibilities,
    required this.objectives,
    required this.reviewDate,
    required this.relatedSwmsIds,
```

```
  }) : super(
    id: id,
    documentType: 'safety_plan',
    title: title,
    description: description,
    createdDate: createdDate,
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
  );
}

class SafetyObjective {
  final String id;
  final String description;
  final String responsibility;
  final DateTime targetDate;
  final bool achieved;

  const SafetyObjective({
    required this.id,
    required this.description,
    required this.responsibility,
    required this.targetDate,
    this.achieved = false,
  });
}

class JSA extends ComplianceDocument {
  final String jobName;
  final String location;
  final List<JobStep> steps;
  final List<Hazard> hazards;
  final List<ControlMeasure> controls;
  final String preparedBy;
  final String approvedBy;

  JSA({
    required String id,
    required String title,
    required String description,
    required DateTime createdDate,
    required String createdBy,
    required DocumentStatus status,
```

```
    required List<AuditEntry> auditTrail,
    required this.jobName,
    required this.location,
    required this.steps,
    required this.hazards,
    required this.controls,
    required this.preparedBy,
    required this.approvedBy,
  }) : super(
    id: id,
    documentType: 'jsa',
    title: title,
    description: description,
    createdDate: createdDate,
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
  );
}

class JobStep {
  final int stepNumber;
  final String description;
  final List<String> hazards;
  final List<String> controls;

  const JobStep({
    required this.stepNumber,
    required this.description,
    required this.hazards,
    required this.controls,
  });
}


PHASE 2: INCIDENT MANAGEMENT & INVESTIGATION (Weeks 6-8)
Incident Reporting, Investigation, Root Cause, Corrective Actions, Trending
[See previous incident section - same content]

PHASE 3: PERMITS, LICENSES & SECURITY CLEARANCES (Weeks 9-11)
Work Permits, Trade Licenses, Security Permits, Background Checks, Clearances

class Permit extends ComplianceDocument {
  final String permitType; // work_permit, trade_license, security_permit, building_permit,
```

```
                    // excavation_permit, confined_space, hot_work, etc
final String permitNumber;
final String issuer;
final String issuedTo;
final DateTime issueDate;
final DateTime expiryDate;
final String documentUrl;
final String? renewalUrl;

// Specific to type
final String? workType; // For work permits
final String? licenseClass; // For trade licenses
final String? scope; // What it covers
final String? restrictions;
final bool requiresRenewal;
final List<String> attachmentUrls;

Permit({
  required String id,
  required String title,
  required String description,
  required DateTime createdDate,
  required DateTime expiryDate,
  required String createdBy,
  required DocumentStatus status,
  required List<AuditEntry> auditTrail,
  required this.permitType,
  required this.permitNumber,
  required this.issuer,
  required this.issuedTo,
  required this.issueDate,
  required this.expiryDate,
  required this.documentUrl,
  this.renewalUrl,
  this.workType,
  this.licenseClass,
  this.scope,
  this.restrictions,
  this.requiresRenewal = true,
  required this.attachmentUrls,
}) : super(
  id: id,
  documentType: 'permit',
  title: title,
```

```
    description: description,
    createdDate: createdDate,
    expiryDate: expiryDate,
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
  );
}

class SecurityClearance extends ComplianceDocument {
  final String clearanceType; // police_check, working_with_children, security_clearance, etc
  final String issuedTo;
  final String issuedBy; // Government agency
  final DateTime issueDate;
  final DateTime expiryDate;
  final String clearanceLevel; // Standard, Intermediate, Secret, Top Secret
  final String referenceNumber;
  final bool requiresRenewal;
  final DateTime? renewalDueDate;

  SecurityClearance({
    required String id,
    required String title,
    required String description,
    required DateTime createdDate,
    required DateTime expiryDate,
    required String createdBy,
    required DocumentStatus status,
    required List<AuditEntry> auditTrail,
    required this.clearanceType,
    required this.issuedTo,
    required this.issuedBy,
    required this.issueDate,
    required this.expiryDate,
    required this.clearanceLevel,
    required this.referenceNumber,
    this.requiresRenewal = true,
    this.renewalDueDate,
  }) : super(
    id: id,
    documentType: 'security_clearance',
    title: title,
    description: description,
    createdDate: createdDate,
```

```
    expiryDate: expiryDate,
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
  );
}

class PermitAndClearanceService extends GetxService {
  final documentRepository = Get.find<DocumentRepository>();
  final auditLogger = Get.find<AuditLoggingService>();
  final voiceEngine = Get.find<VoiceEngine>();
  final signalCore = Get.find<SignalCore>();

  final permits = <Permit>[].obs;
  final clearances = <SecurityClearance>[].obs;
  final expiringDocuments = <ComplianceDocument>[].obs;

  @override
  void onInit() {
    super.onInit();
    loadPermits();
    _setupExpiryChecker();
  }

  Future<void> loadPermits() async {
    final allPermits = await documentRepository.searchDocuments(
      documentType: 'permit',
    );

    final allClearances = await documentRepository.searchDocuments(
      documentType: 'security_clearance',
    );

    // Cast and load
    permits.value = allPermits.whereType<Permit>().toList();
    clearances.value = allClearances.whereType<SecurityClearance>().toList();

    _checkExpiringDocuments();
  }

  void _setupExpiryChecker() {
    Timer.periodic(Duration(days: 1), (_) {
      _checkExpiringDocuments();
    });
```

```
}

void _checkExpiringDocuments() {
  final now = DateTime.now();
  final in30Days = now.add(Duration(days: 30));

  expiringDocuments.clear();

  // Check permits
  expiringDocuments.addAll(
    permits.where((p) => p.expiryDate.isBefore(in30Days) &&
                         p.expiryDate.isAfter(now))
  );

  // Check clearances
  expiringDocuments.addAll(
    clearances.where((c) => c.expiryDate.isBefore(in30Days) &&
                           c.expiryDate.isAfter(now))
  );

  if (expiringDocuments.isNotEmpty) {
    voiceEngine.speak(
      '${expiringDocuments.length} permits or clearances expiring soon'
    );

    // Signal alert
    signalCore.emit(ExpiringDocumentsEvent(expiringDocuments));
  }

  // Check expired
  final expired = permits.where((p) => p.expiryDate.isBefore(now)).toList()
    + clearances.where((c) => c.expiryDate.isBefore(now)).toList();

  if (expired.isNotEmpty) {
    voiceEngine.speak(
      'WARNING: ${expired.length} permits or clearances have EXPIRED'
    );

    signalCore.emit(ExpiredDocumentsEvent(expired.cast<ComplianceDocument>()));
  }
}

Future<void> addPermit({
  required String permitType,
```

```
  required String permitNumber,
  required String issuer,
  required String issuedTo,
  required DateTime issueDate,
  required DateTime expiryDate,
  required String documentUrl,
  String? workType,
  String? scope,
}) async {
  final permit = Permit(
    id: generateId(),
    title: '$permitType: $permitNumber',
    description: 'Issued to $issuedTo by $issuer',
    createdDate: DateTime.now(),
    expiryDate: expiryDate,
    createdBy: getCurrentUserId(),
    status: DocumentStatus.active,
    auditTrail: [],
    permitType: permitType,
    permitNumber: permitNumber,
    issuer: issuer,
    issuedTo: issuedTo,
    issueDate: issueDate,
    expiryDate: expiryDate,
    documentUrl: documentUrl,
    workType: workType,
    scope: scope,
    attachmentUrls: [],
  );

  await documentRepository.saveDocument(permit);

  await auditLogger.logAction(
    documentId: permit.id,
    action: 'permit_added',
    userId: getCurrentUserId(),
    userName: getCurrentUserName(),
    details: '$permitType - $issuedTo',
  );

  permits.add(permit);
  _checkExpiringDocuments();

  voiceEngine.speak('Permit added: $permitNumber');
```

```
  }

  Future<void> addSecurityClearance({
    required String clearanceType,
    required String issuedTo,
    required String clearanceLevel,
    required String referenceNumber,
    required DateTime issueDate,
    required DateTime expiryDate,
  }) async {
    final clearance = SecurityClearance(
      id: generateId(),
      title: '$clearanceType: $issuedTo',
      description: 'Level: $clearanceLevel',
      createdDate: DateTime.now(),
      expiryDate: expiryDate,
      createdBy: getCurrentUserId(),
      status: DocumentStatus.active,
      auditTrail: [],
      clearanceType: clearanceType,
      issuedTo: issuedTo,
      issuedBy: 'Government Agency',
      issueDate: issueDate,
      expiryDate: expiryDate,
      clearanceLevel: clearanceLevel,
      referenceNumber: referenceNumber,
    );

    await documentRepository.saveDocument(clearance);

    await auditLogger.logAction(
      documentId: clearance.id,
      action: 'clearance_added',
      userId: getCurrentUserId(),
      userName: getCurrentUserName(),
      details: '$clearanceType - $issuedTo',
    );

    clearances.add(clearance);
    _checkExpiringDocuments();

    voiceEngine.speak('Security clearance registered: $clearanceType');
  }
}
```

PHASE 4: FLEET & EQUIPMENT MANAGEMENT (Weeks 12-14)
Vehicles, Plant, Equipment, Maintenance Schedules, Compliance, Insurance

```
class FleetAsset extends ComplianceDocument {
  final String assetType; // vehicle, plant, equipment, tool
  final String assetName;
  final String assetClass; // car, truck, forklift, excavator, etc
  final String? registrationNumber; // For vehicles
  final String? serialNumber;
  final String? vin; // Vehicle Identification Number
  final String? manufacturer;
  final String? model;
  final int? yearOfManufacture;

  // Purchase info
  final DateTime purchaseDate;
  final double purchasePrice;

  // Current status
  final String location;
  final AssetStatus status; // active, maintenance, decommissioned

  // Compliance
  final String? registrationExpiry; // Vehicle registration
  final String? rwyExpiry; // Road Worthiness Certificate
  final String? insurancePolicy;
  final DateTime? insuranceExpiryDate;

  // Maintenance
  final List<MaintenanceSchedule> maintenanceSchedules;
  final List<MaintenanceRecord> maintenanceHistory;
  final DateTime? lastMaintenanceDate;
  final DateTime? nextMaintenanceDueDate;
  final int? odometerReading; // For vehicles

  // Depreciation
  final String depreciationMethod;
  final double? residualValue;
  final int usefulLifeYears;

  // Operators/users
  final List<String> authorizedOperators;
```

```
final List<String> requiredCertifications; // Forklift, elevated work, etc

FleetAsset({
  required String id,
  required String title,
  required String description,
  required DateTime createdDate,
  required String createdBy,
  required DocumentStatus status,
  required List<AuditEntry> auditTrail,
  required this.assetType,
  required this.assetName,
  required this.assetClass,
  this.registrationNumber,
  this.serialNumber,
  this.vin,
  this.manufacturer,
  this.model,
  this.yearOfManufacture,
  required this.purchaseDate,
  required this.purchasePrice,
  required this.location,
  required AssetStatus status,
  this.registrationExpiry,
  this.rwyExpiry,
  this.insurancePolicy,
  this.insuranceExpiryDate,
  required this.maintenanceSchedules,
  required this.maintenanceHistory,
  this.lastMaintenanceDate,
  this.nextMaintenanceDueDate,
  this.odometerReading,
  this.depreciationMethod = 'straight_line',
  this.residualValue,
  this.usefulLifeYears = 5,
  required this.authorizedOperators,
  required this.requiredCertifications,
}) : super(
  id: id,
  documentType: 'fleet_asset',
  title: title,
  description: description,
  createdDate: createdDate,
  expiryDate: null,
```

```
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
  );
}

enum AssetStatus {
  active,
  maintenance,
  inspection,
  decommissioned,
  sold,
}

class MaintenanceSchedule {
  final String id;
  final String maintenanceType; // service, inspection, registration renewal, etc
  final String frequency; // monthly, quarterly, annually, or at X hours/km
  final double? frequencyValue; // hours or km
  final DateTime? lastDueDate;
  final DateTime? nextDueDate;
  final String? notes;

  const MaintenanceSchedule({
    required this.id,
    required this.maintenanceType,
    required this.frequency,
    this.frequencyValue,
    this.lastDueDate,
    this.nextDueDate,
    this.notes,
  });
}

class MaintenanceRecord {
  final String id;
  final DateTime maintenanceDate;
  final String maintenanceType;
  final String description;
  final String performedBy;
  final String? vendor;
  final double cost;
  final int? odometerReading;
  final String? notes;
```

```
  final List<String> photoUrls;
  final List<String> invoiceUrls;

  const MaintenanceRecord({
    required this.id,
    required this.maintenanceDate,
    required this.maintenanceType,
    required this.description,
    required this.performedBy,
    this.vendor,
    required this.cost,
    this.odometerReading,
    this.notes,
    required this.photoUrls,
    required this.invoiceUrls,
  });
}

class FleetManagementService extends GetxService {
  final documentRepository = Get.find<DocumentRepository>();
  final auditLogger = Get.find<AuditLoggingService>();
  final voiceEngine = Get.find<VoiceEngine>();
  final signalCore = Get.find<SignalCore>();

  final fleetAssets = <FleetAsset>[].obs;
  final assetsNeedingMaintenance = <FleetAsset>[].obs;
  final assetsWithExpiredCompliance = <FleetAsset>[].obs;

  @override
  void onInit() {
    super.onInit();
    loadFleet();
    _setupMaintenanceChecker();
  }

  Future<void> loadFleet() async {
    final allAssets = await documentRepository.searchDocuments(
      documentType: 'fleet_asset',
    );

    fleetAssets.value = allAssets.whereType<FleetAsset>().toList();
    _checkMaintenance();
  }
```

```
void _setupMaintenanceChecker() {
  Timer.periodic(Duration(days: 1), (_) {
    _checkMaintenance();
  });
}

void _checkMaintenance() {
  assetsNeedingMaintenance.clear();
  assetsWithExpiredCompliance.clear();

  final now = DateTime.now();

  for (final asset in fleetAssets) {
    // Check maintenance due
    if (asset.nextMaintenanceDueDate != null &&
        asset.nextMaintenanceDueDate!.isBefore(now.add(Duration(days: 30)))) {
      assetsNeedingMaintenance.add(asset);
    }

    // Check compliance
    if (asset.registrationExpiry != null &&
        DateTime.parse(asset.registrationExpiry!).isBefore(now)) {
      assetsWithExpiredCompliance.add(asset);
    }

    if (asset.rwyExpiry != null &&
        DateTime.parse(asset.rwyExpiry!).isBefore(now)) {
      assetsWithExpiredCompliance.add(asset);
    }

    if (asset.insuranceExpiryDate != null &&
        asset.insuranceExpiryDate!.isBefore(now)) {
      assetsWithExpiredCompliance.add(asset);
    }
  }

  if (assetsNeedingMaintenance.isNotEmpty) {
    voiceEngine.speak(
      '${assetsNeedingMaintenance.length} fleet assets need maintenance'
    );
    signalCore.emit(FleetMaintenanceRequiredEvent(assetsNeedingMaintenance));
  }

  if (assetsWithExpiredCompliance.isNotEmpty) {
```

```
      voiceEngine.speak(
        'WARNING: ${assetsWithExpiredCompliance.length} fleet assets have EXPIRED
compliance'
      );
      signalCore.emit(FleetComplianceExpiredEvent(assetsWithExpiredCompliance));
    }
  }

  Future<void> registerFleetAsset({
    required String assetType,
    required String assetClass,
    required String assetName,
    required String location,
    required DateTime purchaseDate,
    required double purchasePrice,
    String? registrationNumber,
    String? vin,
    String? manufacturer,
    String? model,
  }) async {
    final asset = FleetAsset(
      id: generateId(),
      title: '$assetClass: $assetName',
      description: 'Purchase: ${purchaseDate.toString().split(' ')[0]}, '
                   'Price: \$$purchasePrice',
      createdDate: DateTime.now(),
      createdBy: getCurrentUserId(),
      status: DocumentStatus.active,
      auditTrail: [],
      assetType: assetType,
      assetName: assetName,
      assetClass: assetClass,
      registrationNumber: registrationNumber,
      manufacturer: manufacturer,
      model: model,
      purchaseDate: purchaseDate,
      purchasePrice: purchasePrice,
      location: location,
      status: AssetStatus.active,
      maintenanceSchedules: _generateDefaultMaintenanceSchedules(assetClass),
      maintenanceHistory: [],
      authorizedOperators: [],
      requiredCertifications: _getRequiredCertifications(assetClass),
    );
```

```
  await documentRepository.saveDocument(asset);

  await auditLogger.logAction(
    documentId: asset.id,
    action: 'fleet_asset_registered',
    userId: getCurrentUserId(),
    userName: getCurrentUserName(),
    details: '$assetClass - $assetName',
  );

  fleetAssets.add(asset);
  _checkMaintenance();

  voiceEngine.speak('Fleet asset registered: $assetName');
}

Future<void> recordMaintenance({
  required String assetId,
  required String maintenanceType,
  required String description,
  required double cost,
  int? odometerReading,
  List<String>? photoUrls,
}) async {
  final asset = fleetAssets.firstWhere((a) => a.id == assetId);

  final record = MaintenanceRecord(
    id: generateId(),
    maintenanceDate: DateTime.now(),
    maintenanceType: maintenanceType,
    description: description,
    performedBy: getCurrentUserName(),
    cost: cost,
    odometerReading: odometerReading,
    photoUrls: photoUrls ?? [],
    invoiceUrls: [],
  );

  asset.maintenanceHistory.add(record);
  asset.lastMaintenanceDate = DateTime.now();

  // Update next maintenance date
  final schedule = asset.maintenanceSchedules
```

```
    .firstWhereOrNull((s) => s.maintenanceType == maintenanceType);

  if (schedule != null) {
    switch (schedule.frequency) {
      case 'monthly':
        asset.nextMaintenanceDueDate = DateTime.now().add(Duration(days: 30));
        break;
      case 'quarterly':
        asset.nextMaintenanceDueDate = DateTime.now().add(Duration(days: 90));
        break;
      case 'annually':
        asset.nextMaintenanceDueDate = DateTime.now().add(Duration(days: 365));
        break;
      default:
        break;
    }
  }

  // Save to database
  await documentRepository.saveDocument(asset);

  await auditLogger.logAction(
    documentId: assetId,
    action: 'maintenance_recorded',
    userId: getCurrentUserId(),
    userName: getCurrentUserName(),
    details: '$maintenanceType - Cost: \$$cost',
  );

  _checkMaintenance();

  voiceEngine.speak('Maintenance recorded for ${asset.assetName}');
}

Future<void> authorizeOperator(String assetId, String operatorId) async {
  final asset = fleetAssets.firstWhere((a) => a.id == assetId);

  if (!asset.authorizedOperators.contains(operatorId)) {
    asset.authorizedOperators.add(operatorId);

    await documentRepository.saveDocument(asset);

    await auditLogger.logAction(
      documentId: assetId,
```

```
      action: 'operator_authorized',
      userId: getCurrentUserId(),
      userName: getCurrentUserName(),
      details: 'Operator: $operatorId',
    );
  }
}

double calculateFleetValue() {
  return fleetAssets.fold<double>(0, (sum, asset) {
    return sum + calculateAssetValue(asset);
  });
}

double calculateAssetValue(FleetAsset asset) {
  final purchasePrice = asset.purchasePrice;
  final residualValue = asset.residualValue ?? purchasePrice * 0.1;
  final depreciableAmount = purchasePrice - residualValue;

  final yearsPassed = DateTime.now().difference(asset.purchaseDate).inDays / 365;

  switch (asset.depreciationMethod) {
    case 'straight_line':
      final annualDepreciation = depreciableAmount / asset.usefulLifeYears;
      final totalDepreciation = annualDepreciation * yearsPassed;
      return max(residualValue, purchasePrice - totalDepreciation);

    case 'declining_balance':
      final rate = 2.0 / asset.usefulLifeYears;
      var value = purchasePrice;
      for (int i = 0; i < yearsPassed.toInt(); i++) {
        value = value * (1 - rate);
      }
      return max(residualValue, value);

    default:
      return purchasePrice;
  }
}

List<MaintenanceSchedule> _generateDefaultMaintenanceSchedules(String assetClass) {
  // Different schedules for different asset types
  switch (assetClass) {
    case 'car':
```

```
    case 'truck':
      return [
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'service',
          frequency: 'quarterly',
        ),
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'registration_renewal',
          frequency: 'annually',
        ),
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'roadworthy_cert',
          frequency: 'annually',
        ),
      ];

    case 'forklift':
    case 'excavator':
      return [
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'inspection',
          frequency: 'monthly',
        ),
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'service',
          frequency: 'quarterly',
        ),
        MaintenanceSchedule(
          id: generateId(),
          maintenanceType: 'certification_renewal',
          frequency: 'annually',
        ),
      ];

    default:
      return [];
  }
}
```

```
  List<String> _getRequiredCertifications(String assetClass) {
    switch (assetClass) {
      case 'forklift':
        return ['Forklift Operation Certificate'];
      case 'elevated_work_platform':
        return ['EWP Operation Certificate'];
      case 'crane':
        return ['Crane Operation Certificate'];
      case 'excavator':
        return ['Heavy Equipment Operation Certificate'];
      default:
        return [];
    }
  }
}
```

PHASE 5: CONTRACTS, INSURANCE & LEGAL DOCUMENTS (Weeks 15-17)
Service Contracts, Employment Contracts, Insurance Policies, Indemnities, Liability

```
class Contract extends ComplianceDocument {
  final String contractType; // employment, service, subcontractor, client, insurance, indemnity
  final String counterpartyName;
  final String? contractValue;
  final DateTime contractStartDate;
  final DateTime? contractEndDate;
  final String contractStatus; // draft, active, expired, terminated

  // Key dates
  final DateTime? renewalDate;
  final int? noticeRequiredDays;

  // Signatories
  final List<Signatory> signatories;
  final String? documentUrl;

  // Key clauses (for compliance checking)
  final bool hasWHSClauses;
  final bool hasInsuranceRequirement;
  final bool hasIPClause;
  final bool hasTerminationClause;
  final bool hasConfidentialityClause;
  final bool hasLiabilityClause;
```

```
// Insurance-specific
final String? policyNumber;
final String? insurer;
final double? coverageAmount;
final String? coverageType; // workers comp, public liability, professional indemnity

Contract({
  required String id,
  required String title,
  required String description,
  required DateTime createdDate,
  DateTime? expiryDate,
  required String createdBy,
  String? approvedBy,
  DateTime? approvedDate,
  required DocumentStatus status,
  required List<AuditEntry> auditTrail,
  String? digitalSignature,
  String? signatureHash,
  required this.contractType,
  required this.counterpartyName,
  this.contractValue,
  required this.contractStartDate,
  this.contractEndDate,
  required this.contractStatus,
  this.renewalDate,
  this.noticeRequiredDays,
  required this.signatories,
  this.documentUrl,
  required this.hasWHSClauses,
  required this.hasInsuranceRequirement,
  required this.hasIPClause,
  required this.hasTerminationClause,
  required this.hasConfidentialityClause,
  required this.hasLiabilityClause,
  this.policyNumber,
  this.insurer,
  this.coverageAmount,
  this.coverageType,
}) : super(
  id: id,
  documentType: 'contract',
  title: title,
  description: description,
```

```
    createdDate: createdDate,
    expiryDate: expiryDate,
    createdBy: createdBy,
    approvedBy: approvedBy,
    approvedDate: approvedDate,
    status: status,
    auditTrail: auditTrail,
    digitalSignature: digitalSignature,
    signatureHash: signatureHash,
  );
}

class Signatory {
  final String id;
  final String name;
  final String role;
  final String? email;
  final bool hasSigned;
  final DateTime? signedDate;
  final String? signatureData;

  const Signatory({
    required this.id,
    required this.name,
    required this.role,
    this.email,
    this.hasSigned = false,
    this.signedDate,
    this.signatureData,
  });
}

class BusinessDocument extends ComplianceDocument {
  final String documentType; // policy, procedure, guideline, template, report
  final String? author;
  final String? reviewer;
  final DateTime lastReviewDate;
  final DateTime? nextReviewDue;
  final List<String> applicableToRoles; // Who it applies to
  final bool requiresAcknowledgement;

  BusinessDocument({
    required String id,
    required String title,
```

```
    required String description,
    required DateTime createdDate,
    required String createdBy,
    required DocumentStatus status,
    required List<AuditEntry> auditTrail,
    required String docType,
    this.author,
    this.reviewer,
    required this.lastReviewDate,
    this.nextReviewDue,
    required this.applicableToRoles,
    this.requiresAcknowledgement = true,
  }) : super(
    id: id,
    documentType: 'business_document',
    title: title,
    description: description,
    createdDate: createdDate,
    createdBy: createdBy,
    status: status,
    auditTrail: auditTrail,
    metadata: {'subType': docType},
  );
}

class ContractManagementService extends GetxService {
  final documentRepository = Get.find<DocumentRepository>();
  final auditLogger = Get.find<AuditLoggingService>();
  final voiceEngine = Get.find<VoiceEngine>();

  final contracts = <Contract>[].obs;
  final businessDocuments = <BusinessDocument>[].obs;
  final contractsNeedingRenewal = <Contract>[].obs;
  final insurancePolicies = <Contract>[].obs;

  @override
  void onInit() {
    super.onInit();
    loadContracts();
    _setupRenewalChecker();
  }

  Future<void> loadContracts() async {
    final allContracts = await documentRepository.searchDocuments(
```

```
    documentType: 'contract',
  );

  contracts.value = allContracts.whereType<Contract>().toList();

  final businessDocs = await documentRepository.searchDocuments(
    documentType: 'business_document',
  );

  businessDocuments.value = businessDocs.whereType<BusinessDocument>().toList();

  _checkRenewalDates();
}

void _setupRenewalChecker() {
  Timer.periodic(Duration(days: 7), (_) {
    _checkRenewalDates();
  });
}

void _checkRenewalDates() {
  final now = DateTime.now();
  final in90Days = now.add(Duration(days: 90));

  contractsNeedingRenewal.value = contracts.where((contract) {
    if (contract.contractEndDate == null) return false;

    return contract.contractEndDate!.isBefore(in90Days) &&
           contract.contractEndDate!.isAfter(now);
  }).toList();

  // Separate insurance policies
  insurancePolicies.value = contracts.where((c) =>
    c.contractType == 'insurance'
  ).toList();

  if (contractsNeedingRenewal.isNotEmpty) {
    voiceEngine.speak(
      '${contractsNeedingRenewal.length} contracts need renewal'
    );
  }
}

Future<void> registerContract({
```

```
  required String contractType,
  required String counterpartyName,
  required String? contractValue,
  required DateTime startDate,
  required DateTime? endDate,
  required List<Signatory> signatories,
  required bool hasWHSClauses,
  required bool hasInsuranceRequirement,
}) async {
  final contract = Contract(
    id: generateId(),
    title: '$contractType: $counterpartyName',
    description: contractValue != null ? 'Value: $contractValue' : '',
    createdDate: DateTime.now(),
    expiryDate: endDate,
    createdBy: getCurrentUserId(),
    status: DocumentStatus.draft,
    auditTrail: [],
    contractType: contractType,
    counterpartyName: counterpartyName,
    contractValue: contractValue,
    contractStartDate: startDate,
    contractEndDate: endDate,
    contractStatus: 'draft',
    signatories: signatories,
    hasWHSClauses: hasWHSClauses,
    hasInsuranceRequirement: hasInsuranceRequirement,
    hasIPClause: true,
    hasTerminationClause: true,
    hasConfidentialityClause: true,
    hasLiabilityClause: true,
  );

  await documentRepository.saveDocument(contract);

  await auditLogger.logAction(
    documentId: contract.id,
    action: 'contract_registered',
    userId: getCurrentUserId(),
    userName: getCurrentUserName(),
    details: '$contractType with $counterpartyName',
  );

  contracts.add(contract);
```

```
  voiceEngine.speak('Contract registered');
}

Future<void> signContract(
  String contractId,
  String signatoryId,
  List<Offset> signaturePoints,
) async {
  final contract = contracts.firstWhere((c) => c.id == contractId);
  final signatory = contract.signatories.firstWhere((s) => s.id == signatoryId);

  // Signature to image
  final signatureData = await _convertSignatureToBase64(signaturePoints);

  signatory.hasSigned = true;
  signatory.signedDate = DateTime.now();
  signatory.signatureData = signatureData;

  // If all signed
  if (contract.signatories.every((s) => s.hasSigned)) {
    contract.contractStatus = 'active';
    contract.status = DocumentStatus.approved;

    // Digital signature
    contract.digitalSignature = await Get.find<DigitalSignatureService>()
      .signDocument(contractId, getCurrentUserId());
  }

  await documentRepository.saveDocument(contract);

  await auditLogger.logAction(
    documentId: contractId,
    action: 'contract_signed',
    userId: getCurrentUserId(),
    userName: getCurrentUserName(),
    details: 'Signed by ${signatory.name}',
  );

  voiceEngine.speak('Contract signed by ${signatory.name}');
}

Future<void> createBusinessDocument({
  required String docType, // policy, procedure, guideline
  required String title,
```

```
    required String content,
    required List<String> applicableRoles,
    required bool requiresAck,
  }) async {
    final doc = BusinessDocument(
      id: generateId(),
      title: title,
      description: content,
      createdDate: DateTime.now(),
      createdBy: getCurrentUserId(),
      status: DocumentStatus.draft,
      auditTrail: [],
      docType: docType,
      author: getCurrentUserName(),
      lastReviewDate: DateTime.now(),
      nextReviewDue: DateTime.now().add(Duration(days: 365)),
      applicableToRoles: applicableRoles,
      requiresAcknowledgement: requiresAck,
    );

    await documentRepository.saveDocument(doc);

    businessDocuments.add(doc);

    voiceEngine.speak('Business document created: $title');
  }

  Future<String> _convertSignatureToBase64(List<Offset> signaturePoints) async {
    // Render signature to image and encode
    return '...';
  }
}
```

PHASE 6: BUSINESS DOCUMENT MANAGEMENT & COMPLIANCE AUTOMATION (Weeks 18-19)
Policy Management, Procedure Documentation, Document Control, Acknowledgements
[Covered above in BusinessDocument class - integrated into Phase 5]

PHASE 7: COMPREHENSIVE COMPLIANCE DASHBOARD & ANALYTICS (Weeks 20-21)
Real-Time Status, OmegaCore Insights, Predictive Alerts, Compliance Scoring

```
class ComplianceDashboardController extends GetxController {
  final swmsService = Get.find<SWMSController>();
```

```
final incidentService = Get.find<IncidentReportingService>();
final permitService = Get.find<PermitAndClearanceService>();
final fleetService = Get.find<FleetManagementService>();
final contractService = Get.find<ContractManagementService>();
final documentRepository = Get.find<DocumentRepository>();
final omegaCore = Get.find<OmegacoreService>();

// Overall metrics
final complianceScore = 0.0.obs;
final riskLevel = RiskLevel.low.obs;
final criticalAlerts = <ComplianceAlert>[].obs;
final insights = <String>[].obs;
final trends = <ComplianceTrend>[].obs;

// Category scores
final swmsScore = 0.0.obs;
final safetyScore = 0.0.obs;
final permitScore = 0.0.obs;
final fleetScore = 0.0.obs;
final contractScore = 0.0.obs;
final documentScore = 0.0.obs;

@override
void onInit() {
  super.onInit();
  calculateComplianceScore();
  _setupRealtimeUpdates();
}

Future<void> calculateComplianceScore() async {
  // Calculate weighted compliance across all categories

  swmsScore.value = _calculateSWMSScore();      // 20%
  safetyScore.value = _calculateSafetyScore();  // 15%
  permitScore.value = _calculatePermitScore();  // 20%
  fleetScore.value = _calculateFleetScore();    // 20%
  contractScore.value = _calculateContractScore(); // 15%
  documentScore.value = _calculateDocumentScore(); // 10%

  complianceScore.value = (swmsScore.value * 0.20) +
                          (safetyScore.value * 0.15) +
                          (permitScore.value * 0.20) +
                          (fleetScore.value * 0.20) +
                          (contractScore.value * 0.15) +
```

```
                    (documentScore.value * 0.10);

    // Determine risk level
    if (complianceScore.value >= 90) {
      riskLevel.value = RiskLevel.low;
    } else if (complianceScore.value >= 75) {
      riskLevel.value = RiskLevel.medium;
    } else if (complianceScore.value >= 60) {
      riskLevel.value = RiskLevel.high;
    } else {
      riskLevel.value = RiskLevel.critical;
    }

    // Get insights from OmegaCore
    insights.value = await omegaCore.getComplianceInsights(
      score: complianceScore.value,
      categories: {
        'SWMS': swmsScore.value,
        'Safety': safetyScore.value,
        'Permits': permitScore.value,
        'Fleet': fleetScore.value,
        'Contracts': contractScore.value,
        'Documents': documentScore.value,
      },
    );

    // Get trends
    trends.value = await omegaCore.getComplianceTrends();

    // Generate alerts
    _generateAlerts();
  }

  double _calculateSWMSScore() {
    if (swmsService.swmsList.isEmpty) return 50; // Deduct if none

    final approved = swmsService.swmsList
      .where((s) => s.status == DocumentStatus.approved)
      .length;

    final expiringSoon = swmsService.swmsList
      .where((s) => s.validUntil.isBefore(DateTime.now().add(Duration(days: 30))))
      .length;
```

```
  var score = (approved / swmsService.swmsList.length) * 100;
  score -= (expiringSoon * 5);

  return max(0, score);
}

double _calculateSafetyScore() {
  // Based on incidents, trends
  final incidents = incidentService.incidents;
  if (incidents.isEmpty) return 100;

  final severeCount = incidents
    .where((i) => i.severity == IncidentSeverity.major ||
                  i.severity == IncidentSeverity.fatality)
    .length;

  final uninvestigated = incidents
    .where((i) => i.status == DocumentStatus.submitted)
    .length;

  var score = 100.0;
  score -= (severeCount * 30);
  score -= (uninvestigated * 10);

  return max(0, score);
}

double _calculatePermitScore() {
  final totalPermits = permitService.permits.length +
                       permitService.clearances.length;

  if (totalPermits == 0) return 50;

  final validPermits = permitService.permits
    .where((p) => p.status == DocumentStatus.active)
    .length +
    permitService.clearances
    .where((c) => c.status == DocumentStatus.active)
    .length;

  return (validPermits / totalPermits) * 100;
}

double _calculateFleetScore() {
```

```
  if (fleetService.fleetAssets.isEmpty) return 50;

  var score = 100.0;

  // Deduct for assets needing maintenance
  score -= (fleetService.assetsNeedingMaintenance.length * 10);

  // Deduct for expired compliance
  score -= (fleetService.assetsWithExpiredCompliance.length * 15);

  return max(0, score);
}

double _calculateContractScore() {
  if (contractService.contracts.isEmpty) return 50;

  final activeContracts = contractService.contracts
    .where((c) => c.status == DocumentStatus.approved)
    .length;

  var score = (activeContracts / contractService.contracts.length) * 100;

  // Check expiring
  score -= (contractService.contractsNeedingRenewal.length * 5);

  return max(0, score);
}

double _calculateDocumentScore() {
  if (contractService.businessDocuments.isEmpty) return 50;

  final upToDate = contractService.businessDocuments
    .where((d) => d.nextReviewDue!.isAfter(DateTime.now()))
    .length;

  return (upToDate / contractService.businessDocuments.length) * 100;
}

void _generateAlerts() {
  criticalAlerts.clear();

  // Critical: Expired permits/clearances
  final expiredPermits = permitService.permits
    .where((p) => p.expiryDate.isBefore(DateTime.now()))
```

```
      .toList();

    if (expiredPermits.isNotEmpty) {
      criticalAlerts.add(ComplianceAlert(
        severity: AlertSeverity.critical,
        title: 'Expired Permits',
        description: '${expiredPermits.length} permits have EXPIRED',
        action: 'Renew immediately',
      ));
    }

    // Critical: Fleet compliance expired
    if (fleetService.assetsWithExpiredCompliance.isNotEmpty) {
      criticalAlerts.add(ComplianceAlert(
        severity: AlertSeverity.critical,
        title: 'Fleet Compliance Expired',
        description: '${fleetService.assetsWithExpiredCompliance.length} vehicles have expired
compliance',
        action: 'Update compliance immediately',
      ));
    }

    // High: Uninvestigated incidents
    final uninvestigatedIncidents = incidentService.incidents
      .where((i) => i.status == DocumentStatus.submitted)
      .toList();

    if (uninvestigatedIncidents.isNotEmpty) {
      criticalAlerts.add(ComplianceAlert(
        severity: AlertSeverity.high,
        title: 'Incidents Pending Investigation',
        description: '${uninvestigatedIncidents.length} incidents awaiting investigation',
        action: 'Start investigation',
      ));
    }

    // Warning: Documents expiring soon
    final expiringDocs = permitService.expiringDocuments;
    if (expiringDocs.isNotEmpty) {
      criticalAlerts.add(ComplianceAlert(
        severity: AlertSeverity.warning,
        title: 'Permits/Clearances Expiring Soon',
        description: '${expiringDocs.length} documents expire in 30 days',
        action: 'Plan renewals',
```

```
      ));
    }
  }

  void _setupRealtimeUpdates() {
    // Recalculate whenever anything changes
    ever(swmsService.swmsList, (_) => calculateComplianceScore());
    ever(incidentService.incidents, (_) => calculateComplianceScore());
    ever(permitService.permits, (_) => calculateComplianceScore());
    ever(fleetService.fleetAssets, (_) => calculateComplianceScore());
    ever(contractService.contracts, (_) => calculateComplianceScore());
  }
}

enum RiskLevel {
  low,
  medium,
  high,
  critical,
}

class ComplianceAlert {
  final AlertSeverity severity;
  final String title;
  final String description;
  final String? action;

  const ComplianceAlert({
    required this.severity,
    required this.title,
    required this.description,
    this.action,
  });
}

enum AlertSeverity {
  info,
  warning,
  high,
  critical,
}

class ComplianceTrend {
  final String category;
```

```
  final List<double> scoreHistory; // Last 12 months
  final double trend; // +5, -2, etc
  final String direction; // improving, declining, stable

  const ComplianceTrend({
    required this.category,
    required this.scoreHistory,
    required this.trend,
    required this.direction,
  });
}

class ComplianceDashboardScreen extends StatelessWidget {
  final controller = Get.put(ComplianceDashboardController());

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(title: Text('Compliance Dashboard')),
      body: Obx(() {
        return SingleChildScrollView(
          padding: EdgeInsets.all(16),
          child: Column(
            children: [
              // Overall Score Card
              Card(
                child: Padding(
                  padding: EdgeInsets.all(16),
                  child: Column(
                    crossAxisAlignment: CrossAxisAlignment.start,
                    children: [
                      Text('Overall Compliance Score',
                        style: TextStyle(fontWeight: FontWeight.bold, fontSize: 16)),
                      SizedBox(height: 12),
                      Row(
                        children: [
                          Expanded(
                            child: LinearProgressIndicator(
                              value: controller.complianceScore.value / 100,
                              minHeight: 10,
                              backgroundColor: Colors.grey[300],
                              valueColor: AlwaysStoppedAnimation(
                                _riskColor(controller.riskLevel.value),
                              ),
```

```
              ),
            ),
            SizedBox(width: 16),
            Text(
              '${controller.complianceScore.value.toStringAsFixed(0)}%',
              style: TextStyle(fontSize: 24, fontWeight: FontWeight.bold),
            ),
          ],
        ),
        SizedBox(height: 12),
        Chip(
          label: Text(controller.riskLevel.value.name.toUpperCase()),
          backgroundColor: _riskColor(controller.riskLevel.value),
        ),
      ],
    ),
  ),
),

SizedBox(height: 16),

// Category Scores
GridView.count(
  crossAxisCount: 2,
  shrinkWrap: true,
  physics: NeverScrollableScrollPhysics(),
  mainAxisSpacing: 8,
  crossAxisSpacing: 8,
  children: [
    _buildCategoryCard('SWMS', controller.swmsScore.value),
    _buildCategoryCard('Safety', controller.safetyScore.value),
    _buildCategoryCard('Permits', controller.permitScore.value),
    _buildCategoryCard('Fleet', controller.fleetScore.value),
    _buildCategoryCard('Contracts', controller.contractScore.value),
    _buildCategoryCard('Documents', controller.documentScore.value),
  ],
),

SizedBox(height: 16),

// Critical Alerts
if (controller.criticalAlerts.isNotEmpty)
  Card(
    color: Colors.red[50],
```

```
    child: Padding(
      padding: EdgeInsets.all(12),
      child: Column(
        crossAxisAlignment: CrossAxisAlignment.start,
        children: [
          Text(
            '🚨 ${controller.criticalAlerts.length} Alerts',
            style: TextStyle(
              fontWeight: FontWeight.bold,
              color: Colors.red[900],
            ),
          ),
          SizedBox(height: 12),
          ...controller.criticalAlerts.map((alert) => Padding(
            padding: EdgeInsets.only(bottom: 8),
            child: Column(
              crossAxisAlignment: CrossAxisAlignment.start,
              children: [
                Text(alert.title, style: TextStyle(fontWeight: FontWeight.bold)),
                Text(alert.description, style: TextStyle(fontSize: 12)),
                if (alert.action != null)
                  ElevatedButton(
                    onPressed: () {},
                    style: ElevatedButton.styleFrom(
                      backgroundColor: Colors.red,
                    ),
                    child: Text(alert.action!),
                  ),
              ],
            ),
          )),
        ],
      ),
    ),
  ),

SizedBox(height: 16),

// Insights
if (controller.insights.isNotEmpty)
  Card(
    child: Padding(
      padding: EdgeInsets.all(12),
      child: Column(
```

```
                crossAxisAlignment: CrossAxisAlignment.start,
                children: [
                  Text('💡 Insights', style: TextStyle(fontWeight: FontWeight.bold)),
                  SizedBox(height: 12),
                  ...controller.insights.map((insight) => Padding(
                    padding: EdgeInsets.only(bottom: 8),
                    child: Text('• $insight', style: TextStyle(fontSize: 12)),
                  )),
                ],
              ),
            ),
          ),
        ],
      ),
    );
  }),
 );
}

Widget _buildCategoryCard(String category, double score) {
 return Card(
  child: Padding(
   padding: EdgeInsets.all(12),
   child: Column(
    mainAxisAlignment: MainAxisAlignment.center,
    children: [
     Text(category, style: TextStyle(fontWeight: FontWeight.bold)),
     SizedBox(height: 8),
     Text(
      '${score.toStringAsFixed(0)}%',
      style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold),
     ),
     SizedBox(height: 8),
     LinearProgressIndicator(value: score / 100),
    ],
   ),
  ),
 );
}

Color _riskColor(RiskLevel level) {
 return {
  RiskLevel.low: Colors.green,
  RiskLevel.medium: Colors.amber,
```

```
      RiskLevel.high: Colors.orange,
      RiskLevel.critical: Colors.red,
    }[level] ?? Colors.grey;
  }
}
```

PHASE 8: TESTING, ACCESSIBILITY & DEPLOYMENT (Week 22)
Complete Testing, WCAG Compliance, Beta, Production Launch

Testing:
├─ Unit tests (250+ tests)
├─ Integration tests (80+ tests)
├─ Widget tests (120+ tests)
└─ Target: >80% code coverage

Accessibility (WCAG 2.1 AA):
├─ Screen reader compatible
├─ Large text support
├─ High contrast mode
├─ Voice announcements
└─ Keyboard navigation

Beta: 30-50 safety professionals
Deployment: Staged rollout

FINAL STATISTICS

TOTAL TIME: 22 weeks (5.5 months)
TEAM: 3 devs + 1 designer + 1 QA

CODE: 28,000-32,000 LOC | 400+ tests

DOCUMENT TYPES: 8+
├─ SWMS, JSA, Safety Plans
├─ Incidents, Investigations
├─ Permits, Clearances, Licenses
├─ Fleet Assets, Equipment, Vehicles
├─ Contracts, Insurance, Indemnities
└─ Business Documents, Policies

FEATURES:
✅ Unified document management (all types)

✅ Real-time compliance scoring (6 categories)
✅ Risk assessment & trending
✅ Critical alerts & notifications
✅ Digital signatures (all documents)
✅ Audit trails (complete, non-repudiation)
✅ Encryption at rest + transit
✅ Cloud storage integration (user's API)
✅ Offline-first capability
✅ OmegaCore insights & forecasting
✅ Voice commands (all major features)
✅ Fleet management + maintenance
✅ Insurance & contract tracking
✅ Business document control
✅ Multi-user collaboration
✅ Expiry tracking & alerts
✅ Renewal workflows
✅ Analytics & dashboards

SECURITY:
├─ AES-256 encryption at rest
├─ TLS 1.3 in transit
├─ Digital signatures (SHA-256)
├─ Audit logging (complete)
├─ Non-repudiation (all changes)
├─ Zero hard-coded secrets
└─ Privacy-first (no Titan servers)

COMPLIANCE:
├─ Australian work health & safety
├─ Risk management frameworks
├─ Incident investigation
├─ Permit & license management
├─ Fleet management standards
├─ Contract management best practices
├─ Document control
├─ Regulatory scoring
└─ Real-time alert system

INTEGRATIONS:
├─ Voice (text-to-speech)
├─ Maps (location tracking)
├─ Cloud storage (user's API: S3, Azure, GCP)
├─ Email (user's API: SendGrid, AWS SES)
├─ Payment (for insurance, renewals)

└─ Sync (federated database)

PERFORMANCE:
├─ Startup: < 2 seconds
├─ Document ops: < 300ms
├─ Sync: < 500ms
├─ Score calc: < 1 second
├─ Memory: < 200MB
└─ Battery: Minimal impact

ACCESSIBILITY:
├─ WCAG 2.1 AA compliant
├─ Screen reader support
├─ Keyboard navigation (all features)
├─ Large text (up to 200%)
├─ High contrast mode
├─ Voice commands (all major tasks)
└─ Haptic feedback on alerts